

PHU NGUYEN

Module 04 Lab - Exploratory Data Analysis **Objective:** To learn how to explore a dataset to find patterns, anomalies, and insights before modeling. EDA is like being a detective; you are looking for clues in the data that will help you build a better model. **This lab is fully coded.** Your task is to run each cell, read the detailed explanations, understand the purpose of each visualization, and then complete the experimentation section.

Part 1: Setup and Data Loading ****What is Exploratory Data Analysis (EDA)?**** EDA is the process of using summary statistics and visualizations to understand a dataset's main characteristics. Before you can build a model, you need to understand your data. What stories does it tell? Are there errors or missing values? Are there strong relationships between variables? EDA helps answer these questions. We will use the famous Titanic dataset for this lab. It contains information about passengers and, crucially, whether they survived the disaster.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns # Seaborn is a library built on top of Matplotlib that makes creating beautiful plots easier.

# Load the dataset directly from a URL
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')
print("--- First 5 Rows ---")
print(df.head())
print("--- Basic Info ---")
# .info() is a great first command. It tells us the column names, how many non-null values are in each column, and the
# Notice that 'Age' and 'Cabin' have missing values!
df.info()
```

```
--- First 5 Rows ---
  PassengerId  Survived  Pclass \
0             1         0       3
1             2         1       1
2             3         1       3
3             4         1       1
4             5         0       3

      Name               Sex  Age  SibSp \
0  Braund, Mr. Owen Harris   male  22.0    1
1  Cumings, Mrs. John Bradley (Florence Briggs Th... female  38.0    1
2    Heikkinen, Miss. Laina  female  26.0    0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)    female  35.0    1
4    Allen, Mr. William Henry   male  35.0    0

   Parch    Ticket   Fare Cabin Embarked
0      0      A/5 21171   7.2500   NaN      S
1      0      PC 17599  71.2833   C85      C
2      0  STON/O2. 3101282   7.9250   NaN      S
3      0    113803   53.1000  C123      S
4      0    373450   8.0500   NaN      S

--- Basic Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column             Non-Null Count  Dtype
---  -
0   PassengerId         891 non-null    int64
1   Survived            891 non-null    int64
```

```

2  Pclass      891 non-null    int64
3  Name        891 non-null    object
4  Sex         891 non-null    object
5  Age         714 non-null    float64
6  SibSp       891 non-null    int64
7  Parch       891 non-null    int64
8  Ticket      891 non-null    object
9  Fare        891 non-null    float64
10 Cabin       204 non-null    object
11 Embarked    889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

```

Part 2: Descriptive Statistics Let's start by getting a high-level numerical summary of the data. The `.describe()` method is perfect for this. It calculates statistics like mean, standard deviation, min, and max for the numerical columns.

```
# Get summary statistics for numerical columns
print("--- Descriptive Statistics ---")
print(df.describe())
print("--- K
```

Part 3: Visual EDA - Telling Stories with Plots Numbers are great, but plots make patterns and relationships immediately obvious. The goal of visual EDA is to turn data into insights. **A Note on Plotting Libraries:** * **Matplotlib:** The foundational library, gives you full control over everything. * **Seaborn:** Built on Matplotlib, it provides a simpler, high-level interface for creating common statistical plots. We will use Seaborn for its ease of use and attractive defaults.

Visualization 1: How many survived? A simple `countplot` is the best way to see the distribution of a categorical variable, like our target `Survived`.

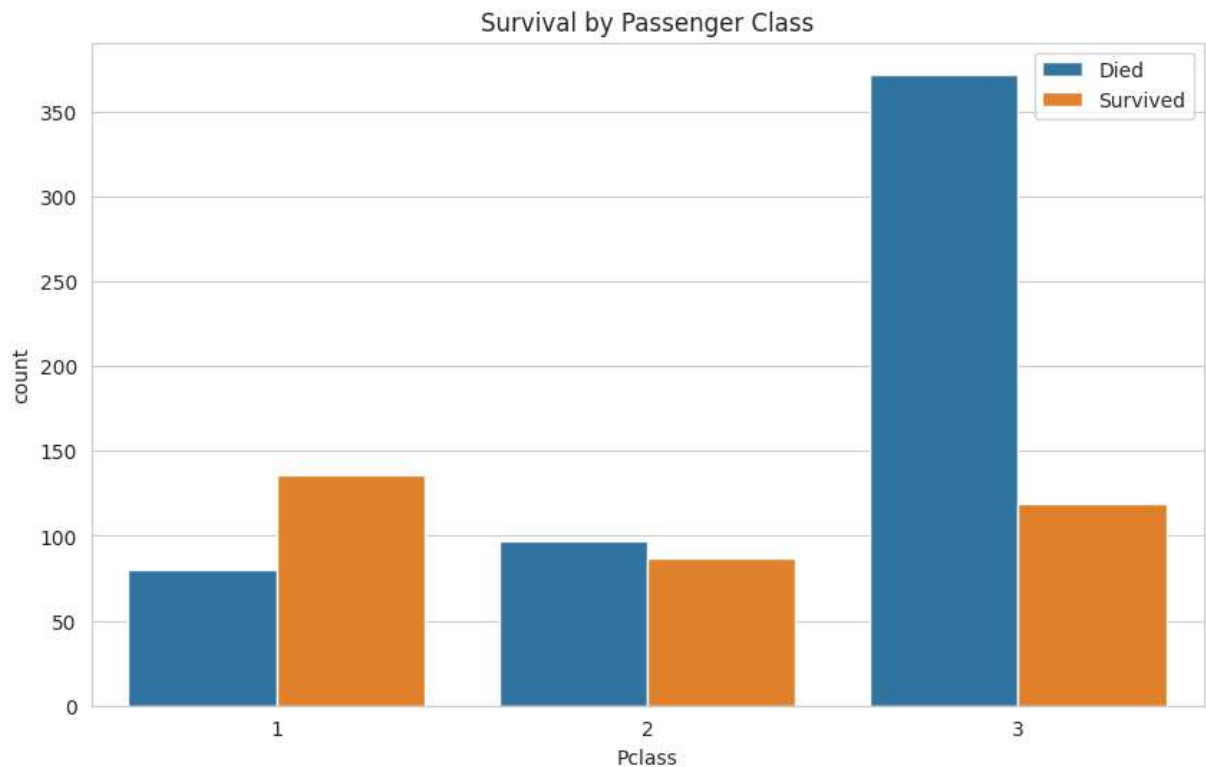
```
sns.set_style('whitegrid') # Sets a nice visual style for our plots
plt.figure(figsize=(8, 6))
sns.countplot(x='Survive
```

Visualization 2: Does passenger class matter for survival? Now we want to see if there's a relationship between two variables. We can use the `hue` parameter in `countplot` to split the bars by another category.

```

plt.figure(figsize=(10, 6))
sns.countplot(x='Pclass', hue='Survived', data=df)
plt.title('Survival by Passenger Class')
plt.legend(['Died', 'Survived'])
plt.show()
print("Insight: This is a very strong pattern. 1st class passengers had a much higher chance of survival compared to 3r

```

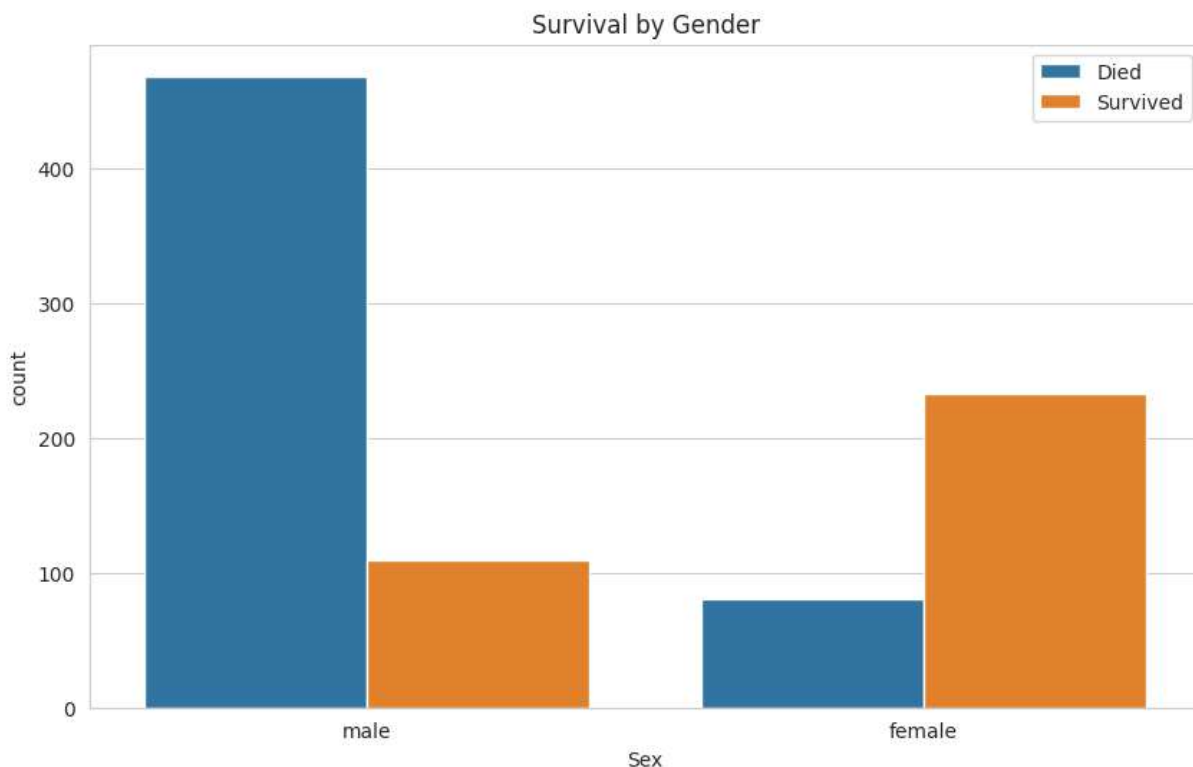


Insight: This is a very strong pattern. 1st class passengers had a much higher chance of survival compared to 3rd class passengers.

v

Visualization 3: What about gender? The 'women and children first' mantra is famous. Let's see if the data supports it.

```
plt.figure(figsize=(10, 6))
sns.countplot(x='Sex', hue='Survived', data=df)
plt.title('Survival by Gender')
plt.legend(['Died', 'Survived'])
plt.show()
print("Insight: The pattern is undeniable. A much higher proportion of females survived compared to males. This is another strong pattern.")
```



Insight: The pattern is undeniable. A much higher proportion of females survived compared to males. This is another ve

- Visualization 4: How does age play a role? For a continuous variable like `Age`, a histogram is a great way to see its distribution.

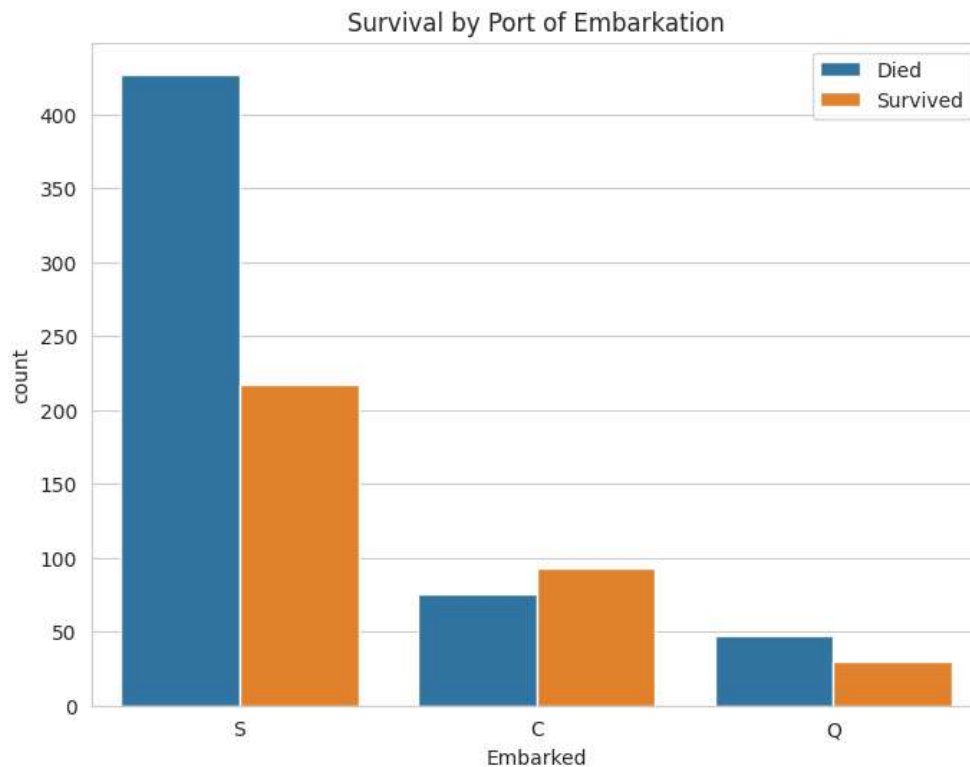
A FacetGrid allows us to create multiple plots side-by-side to compare distributions. # Here, we create one histogram

- Part 4: Student Experimentation **Instructions:** Create your own visualizations to explore other relationships in the data.

- Experiment 1: Port of Embarkation 1. The `Embarked` column tells you where a passenger boarded the ship (C = Cherbourg, Q = Queenstown, S = Southampton). 2. Create a `countplot` to see how survival rates differed by the port of embarkation. Does where they boarded seem to be related to their survival?

```
# --- ENTER YOUR CODE HERE ---
plt.figure(figsize=(8, 6))
sns.countplot(x='Embarked', hue='Survived', data=df)
plt.title('Survival by Port of Embarkation')
plt.legend(['Died', 'Survived'])
plt.show()

# survival rate per port
print(df.groupby('Embarked')['Survived'].mean())
```



```
Embarked
C    0.553571
Q    0.389610
S    0.336957
Name: Survived, dtype: float64
```

- Experiment 2: Fare vs. Survival1. Fare is a continuous numerical variable.2. A boxplot or violinplot is a great way to see the distribution of fares for those who survived vs. those who didn't.3. Create one of these plots to compare the Fare distribution by Survived. What does it tell you?

```
# --- ENTER YOUR CODE HERE ---
plt.figure(figsize=(8, 6))
sns.boxplot(x='Survived', y='Fare', data=df)
plt.title('Fare by Survival (0 = Died, 1 = Survived)')
plt.show()

print(df.groupby('Survived')['Fare'].describe())
```


CYNDY NDONG

Module 04 Lab - Exploratory Data Analysis **Objective:** To learn how to explore a dataset to find patterns, anomalies, and insights before modeling. EDA is like being a detective; you are looking for clues in the data that will help you build a better model. **This lab is fully coded.** Your task is to run each cell, read the detailed explanations, understand the purpose of each visualization, and then complete the experimentation section.

Part 1: Setup and Data Loading ****What is Exploratory Data Analysis (EDA)?**** EDA is the process of using summary statistics and visualizations to understand a dataset's main characteristics. Before you can build a model, you need to understand your data. What stories does it tell? Are there errors or missing values? Are there strong relationships between variables? EDA helps answer these questions. We will use the famous Titanic dataset for this lab. It contains information about passengers and, crucially, whether they survived the disaster.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset directly from a URL
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

print("--- First 5 Rows ---")
print(df.head())

print("--- Info ---")
df.info()
```

```
--- First 5 Rows ---
   PassengerId  Survived  Pclass \
0             1         0       3
1             2         1       1
2             3         1       3
3             4         1       1
4             5         0       3

   Name                               Sex  Age  SibSp \
0  Braund, Mr. Owen Harris             male  22.0    1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0    1
2    Heikkinen, Miss. Laina             female  26.0    0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)     female  35.0    1
4    Allen, Mr. William Henry             male  35.0    0

   Parch  Ticket   Fare Cabin Embarked
0      0    A/5 21171   7.2500   NaN      S
1      0      PC 17599  71.2833   C85      C
2      0  STON/O2. 3101282   7.9250   NaN      S
3      0    113803  53.1000  C123      S
4      0    373450   8.0500   NaN      S

--- Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age         714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
```

```

9   Fare      891 non-null   float64
10  Cabin     204 non-null   object
11  Embarked  889 non-null   object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

```

Part 2: Descriptive Statistics Let's start by getting a high-level numerical summary of the data.

- ✓ The `.describe()` method is perfect for this. It calculates statistics like mean, standard deviation, min, and max for the numerical columns.

```
# Get summary statistics for numerical columns
print("--- Descriptive Statistics ---")
print(df.describe())
print("--- Key Insight ---")
```

Part 3: Visual EDA - Telling Stories with Plots Numbers are great, but plots make patterns and relationships immediately obvious. The goal of visual EDA is to turn data into insights. **A Note on**

- ✓ **Plotting Libraries:**
 - * **Matplotlib:** The foundational library, gives you full control over everything.*
 - * **Seaborn:** Built on Matplotlib, it provides a simpler, high-level interface for creating common statistical plots. We will use Seaborn for its ease of use and attractive defaults.

- ✓ Visualization 1: How many survived? A simple `countplot` is the best way to see the distribution of a categorical variable, like our target `Survived`.

```
sns.set_style('whitegrid') # Sets a nice visual style for our plots
plt.figure(figsize=(8, 6))
sns.countplot(x='Survived', data=d)
```

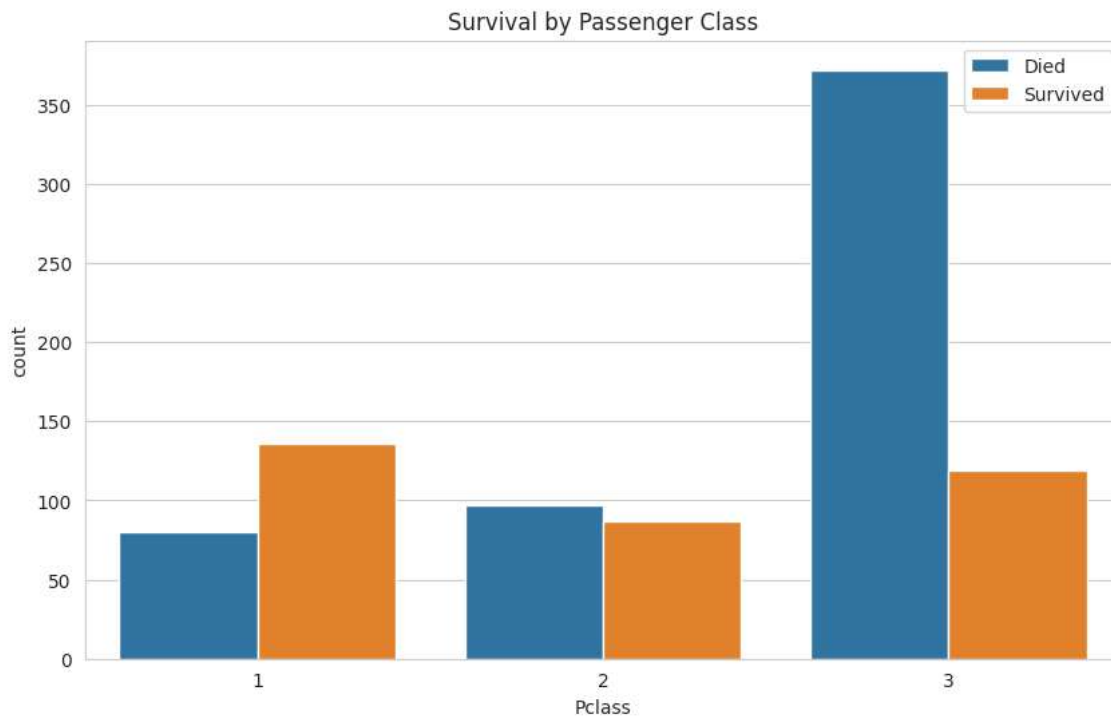
- ✓ Visualization 2: Does passenger class matter for survival? Now we want to see if there's a relationship between two variables. We can use the `hue` parameter in `countplot` to split the bars by another category.

```

plt.figure(figsize=(10, 6))
sns.countplot(x='Pclass', hue='Survived', data=df)
plt.title('Survival by Passenger Class')
plt.legend(['Died', 'Survived'])
plt.show()

```

```
print("Insight: This is a very strong pattern. 1st class passengers had a much higher chance of survival compared to 3rd class")
```



Insight: This is a very strong pattern. 1st class passengers had a much higher chance of survival compared to 3rd class passengers.

- Visualization 3: What about gender? The 'women and children first' mantra is famous. Let's see if the data supports it.

```
plt.figure(figsize=(10, 6))sns.countplot(x='Sex', hue='Survived', data=df)plt.title('Survival by Gender')plt.legend(['Died', 'S
```

```
File "/tmp/ipykernel_874/1436840290.py", line 1
plt.figure(figsize=(10, 6))sns.countplot(x='Sex', hue='Survived', data=df)plt.title('Survival by
Gender')plt.legend(['Died', 'Survived'])plt.show()print("Insight: The pattern is undeniable. A much higher proportion of
females survived compared to males. This is another very strong predictor.")
SyntaxError: invalid syntax
```

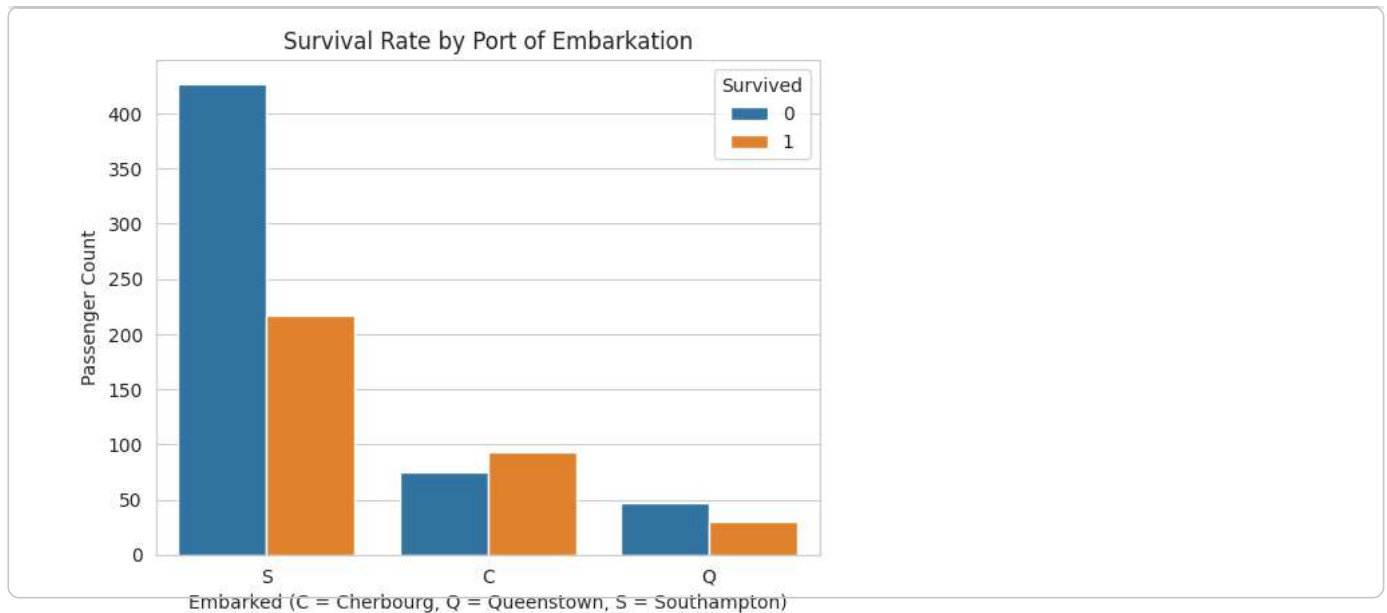
- Visualization 4: How does age play a role? For a continuous variable like `Age`, a histogram is a great way to see its distribution.

```
# A FacetGrid allows us to create multiple plots side-by-side to compare distributions.# Here, we create one histogram for pass
```

- Part 4: Student Experimentation **Instructions:** Create your own visualizations to explore other relationships in the data.

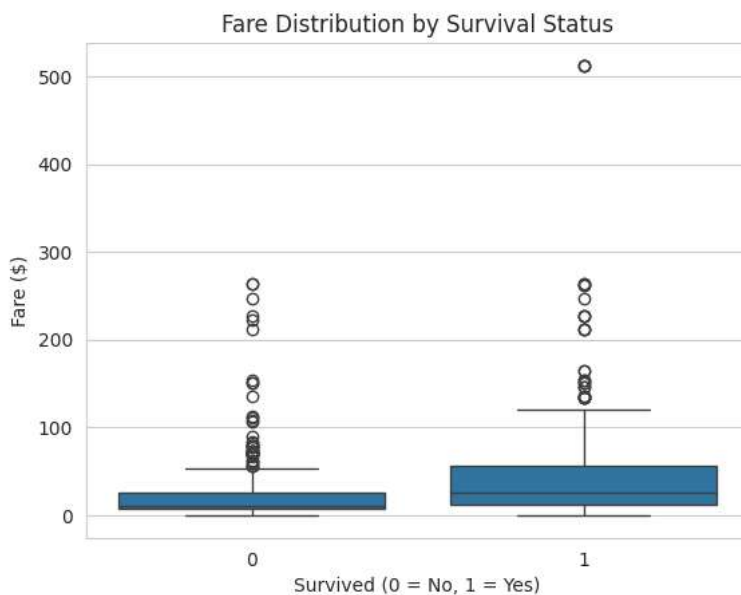
- Experiment 1: Port of Embarkation1. The `Embarked` column tells you where a passenger boarded the ship (C = Cherbourg, Q = Queenstown, S = Southampton).2. Create a `countplot` to see how survival rates differed by the port of embarkation. Does where they boarded seem to be related to their survival?

```
# Create a countplot to see survival rates by port of embarkation
sns.countplot(data=df, x='Embarked', hue='Survived')
plt.title('Survival Rate by Port of Embarkation')
plt.xlabel('Embarked (C = Cherbourg, Q = Queenstown, S = Southampton)')
plt.ylabel('Passenger Count')
plt.show()
```



- Experiment 2: Fare vs. Survival1. Fare is a continuous numerical variable.2. A `boxplot` or `violinplot` is a great way to see the distribution of fares for those who survived vs. those who didn't.3. Create one of these plots to compare the Fare distribution by Survived. What does it tell you?

```
# Create a boxplot to compare fare distribution by survival status
sns.boxplot(data=df, x='Survived', y='Fare')
plt.title('Fare Distribution by Survival Status')
plt.xlabel('Survived (0 = No, 1 = Yes)')
plt.ylabel('Fare ($)')
plt.show()
```



Knowledge CheckInstructions: Answer the following questions in this markdown cell.1.

What is the primary goal of Exploratory Data Analysis (EDA)?2. Based on the plots in this